

Python - Private Variables

Python is an object-oriented programming language that allows you to create classes and objects. Encapsulation is one of the core principles of OPPs. To achieve **encapsulation**, we use private variables and methods. In this chapter, we will discuss private variables in Python and how to use them.

Following topics will be covered in this chapter –

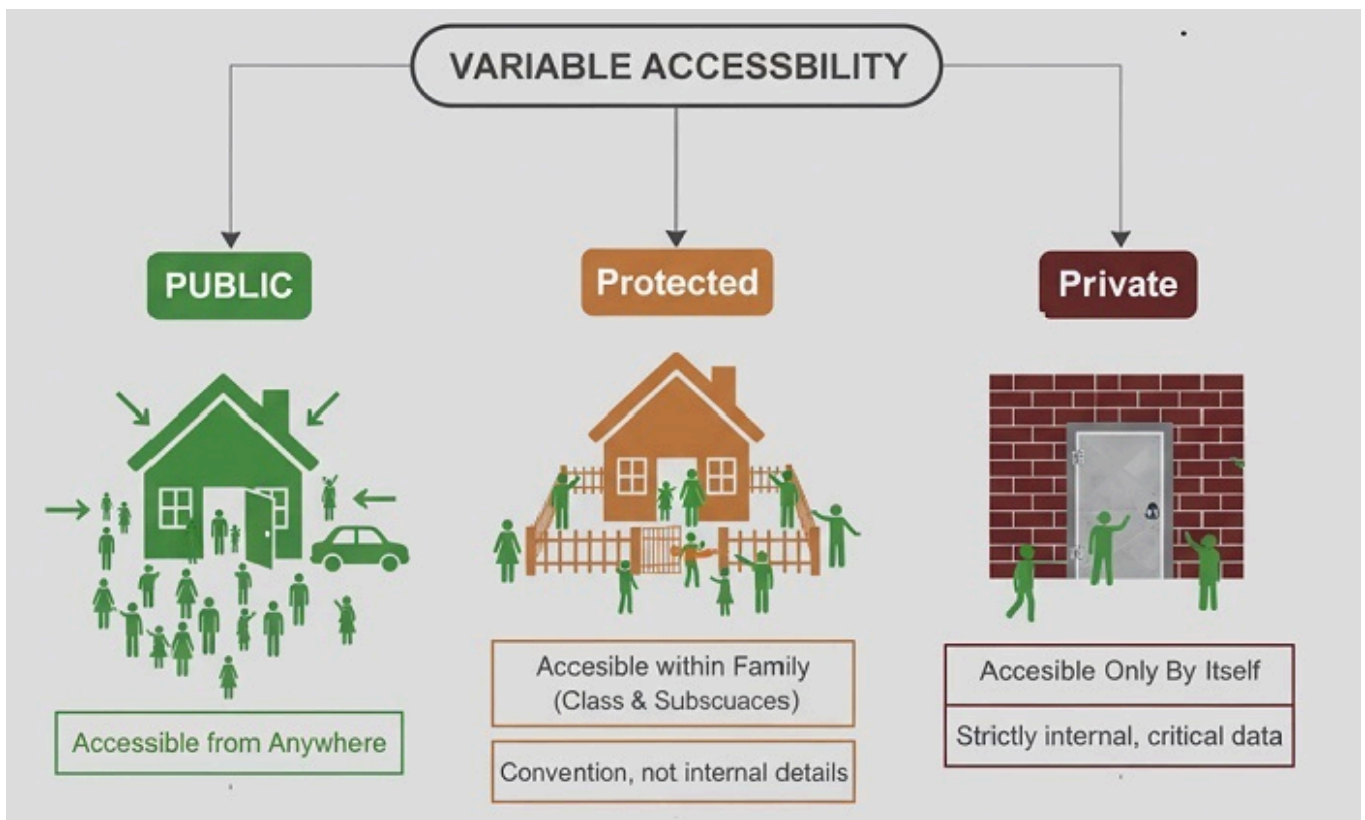
- What are Private Variables?
- Defining Private Variables in Python
- Name Mangling for Private Variables
- Private Methods in Python
- Real World Example of Private Variables
- Public vs Protected vs Private Variables

What are Private Variables?

Private variables are a special type of variable that is only accessible within the class they are defined in. Means, they are not directly accessible from outside the class or from any subclass. This is done to protect the data and ensure that it is not modified or accessed by unauthorized code snippets.

Only direct access is restricted. We can define a public method to access the private variable from outside the class, it is called a **getter method**. Similarly, we can use a **setter method** to modify the value of a private variable from outside the class.

The image shows how Private variables are different from Public and Protected variables.



- **Public Variables:** Public variables are accessible from anywhere, both inside and outside the class. In the image, a public place is considered as an analogy for public variables as they can be accessed by anyone.
- **Protected Variables:** Protected variable can be accessed within the class and its subclasses. In the image, a private domicile is considered as an analogy for protected variables as they can be accessed by family members (subclasses) but not by outsiders.
- **Private Variables:** Private variables are only accessible within the class they are defined in. In the image, a vault is considered as an analogy for private variables as they can only be accessed by the owner (the class itself) and not by anyone else.

Defining Private Variables in Python

In Python, private variables are defined by adding a **double underscore (__)** before the variable name. Unlike other programming languages like Java or C++, Python does not have strict access modifiers like public, private, or protected. So, the double underscore is a convention used to indicate that a variable is private. Similarly, a single underscore (__) is used to indicate that a variable is protected.

Example

The code below shows how to define a private variable and access it using a public method.



```
class MyClass:
    def __init__(self):
        self.__private_var = "I am Private"

    def show_private(self):
        return self.__private_var

obj = MyClass()
# print(obj.__private_var)    # X AttributeError
print(obj.show_private())    # ✓ Access through method
```

We have commented out the direct access to the private variable (Uncommenting it will raise an **AttributeError**). So, the output of the above code will be –

```
I am Private
```

Name Mangling for Private Variables

In Python, private variables are not truly private. They can still be accessed from outside the class using a technique called **name mangling**. That is, when you create a private variable using a double underscore (`__var`), Python internally changes its name to `__ClassName__var`. Now, you can access the private variable using this mangled name. Let's see an example to understand this better.



```
class MyClass:
    def __init__(self):
        self.__private_var = "I am Private"

    def show_private(self):
        return self.__private_var

obj = MyClass()
# Accessing private variable using name mangling
print(obj._MyClass__private_var) # ✓ Access using name mangling
```

The **output** of the above code will be –

I am Private

Private Methods in Python

Similar to private variables, we can also define private methods in Python. **Private methods** are methods that are only accessible within the class they are defined in. They are not accessible from outside the class or from any subclass. To define a private method, we use a double underscore (__) before the method name.

```
class MyClass:
    def __init__(self):
        self.__private_var = "I am Private"

    def __private_method(self):
        return "This is a private method"

    def show_private(self):
        return self.__private_var + " and " + self.__private_method()

obj = MyClass()
print(obj.show_private())    # ✓ Access through method
# print(obj.__private_method()) # ✗ AttributeError
print(obj._MyClass__private_method()) # ✓ Access using name mangling
```

The **output** of the above code will be –

I am Private and This is a private method
This is a private method

Real World Example of Private Variables

Consider a banking application where we have a class called **BankAccount**. This class has private variables for the account number and balance. We will define public methods to deposit, withdraw, and check the balance. We need to ensure that balance cannot be directly modified outside the class, but can only be changed through deposit and withdraw methods.

The code below demonstrates this concept.

```

class BankAccount:
    def __init__(self, account_number, balance):
        self.__account_number = account_number    # Private
        self.__balance = balance                  # Private

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
        return self.__balance

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
        return self.__balance

    def get_balance(self):
        return self.__balance

account = BankAccount("12345", 1000)

# Direct access will fail
try:
    account.__balance += 500 # X AttributeError
except AttributeError:
    print("Direct access to private variable failed!!!")

# Access using methods
print("Your account balance is: ", account.get_balance())    # ✓ 1000

account.deposit(500)
print("Your account balance after deposit is: ", account.get_balance()) # ✓
1500

```

The **output** of the above code will be –

```

Direct access to private variable failed!!!
Your account balance is:  1000
Your account balance after deposit is:  1500

```

Public vs Protected vs Private Variables

Here is a tabular comparison of Public, Protected, and Private variables in Python –

Feature	Public Variables	Protected Variables	Private Variables
Definition	Variables that can be accessed from anywhere.	Variables that can be accessed within the class and its subclasses.	Variables that can only be accessed within the class they are defined in.
Syntax	<code>int var</code>	<code>int _var</code>	<code>int __var</code>
Security	Least secure	Moderately secure	Most secure
Example	<code>int age = 25</code>	<code>int _age = 25</code>	<code>int __age = 25</code>

Conclusion

In this chapter, we learned about private variables in Python and how to use them to achieve encapsulation. We also discussed name mangling, how private methods work, getters and setters. The name mangling technique allows us to access private variables and methods from outside the class, but it is not recommended to use it as it breaks the encapsulation principle.

TOP TUTORIALS

[Python Tutorial](#)
[Java Tutorial](#)
[C++ Tutorial](#)
[C Programming Tutorial](#)
[C# Tutorial](#)
[PHP Tutorial](#)
[R Tutorial](#)
[HTML Tutorial](#)
[CSS Tutorial](#)
[JavaScript Tutorial](#)
[SQL Tutorial](#)

TRENDING TECHNOLOGIES

Cloud Computing Tutorial
Amazon Web Services Tutorial
Microsoft Azure Tutorial
Git Tutorial
Ethical Hacking Tutorial
Docker Tutorial
Kubernetes Tutorial
DSA Tutorial
Spring Boot Tutorial
SDLC Tutorial
Unix Tutorial

CERTIFICATIONS

Business Analytics Certification
Java & Spring Boot Advanced Certification
Data Science Advanced Certification
Cloud Computing And DevOps



Chapters ▾

Categories ≡

DevOps Certification
Game Development Certification
Front-End Developer Certification
AWS Certification Training
Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler
Online Python Compiler
Online Go Compiler
Online C Compiler
Online C++ Compiler
Online C# Compiler
Online PHP Compiler
Online MATLAB Compiler
Online Bash Terminal
Online SQL Compiler

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2026. All Rights Reserved.